Agent Loops & Multi-Tool Orchestration

Unit 2 — Part 13: From Single Tools to Autonomous Agents

The Single Tool Limitation

Simple Task

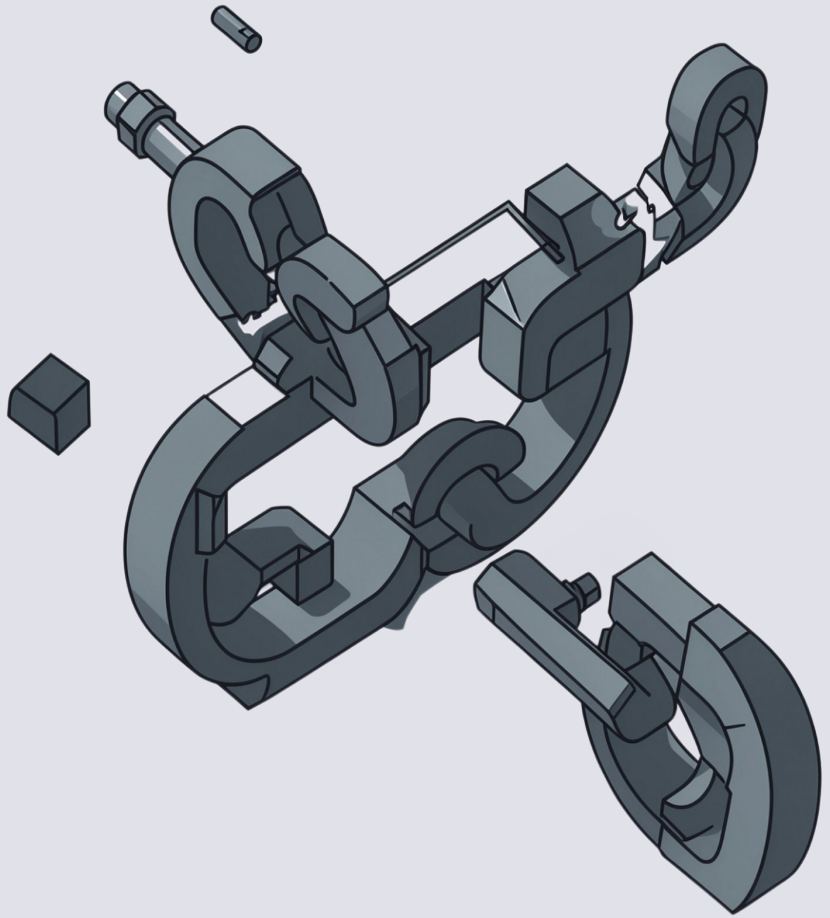
User → LLM → Tool → Result → Answer

One call, one result. Clean and complete.

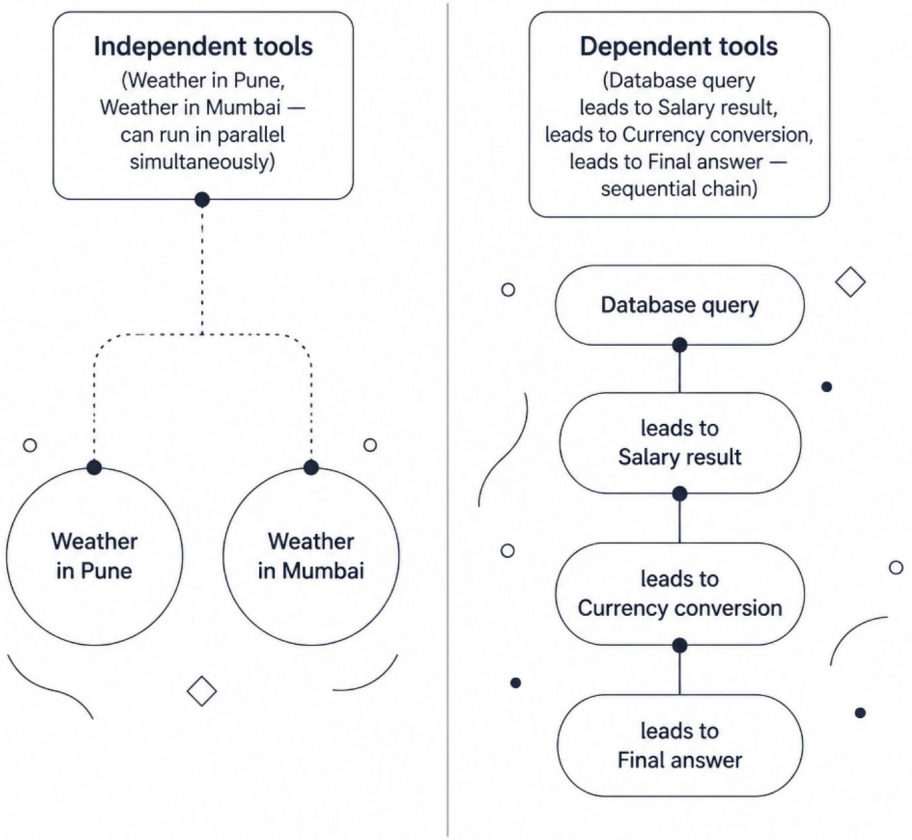
Real Task

"Find top product, calculate revenue in USD, search market info"

- One tool call isn't enough
- Output of Tool A becomes input for Tool B
- Sequential dependencies require an orchestration layer



Why Dependencies Matter



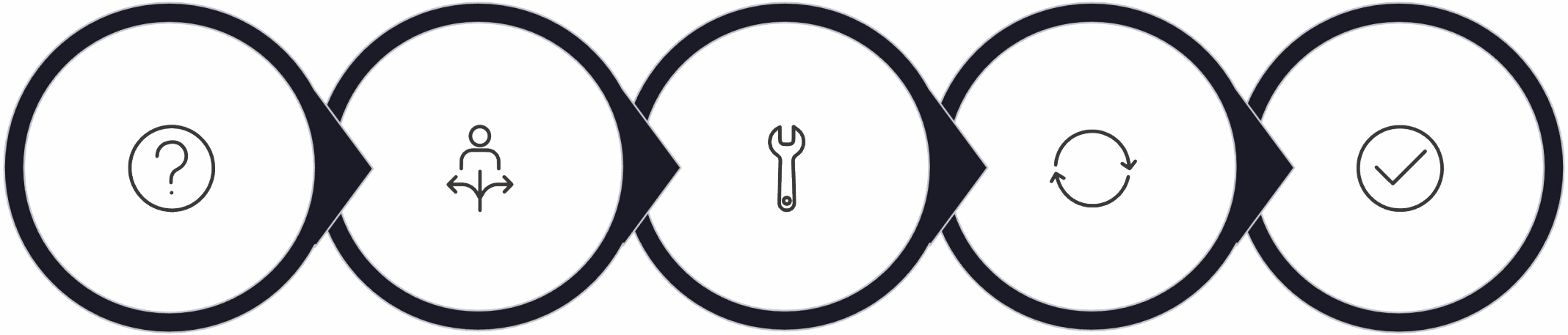
Planning Determines Execution Order

Not all tool calls are equal. The agent must understand **which tools depend on prior results** before deciding how to execute.

- **Independent tools** can run in parallel — no waiting
- **Dependent tools** must execute in sequence — order matters
- Wrong ordering = wrong answer or runtime failure

This dependency analysis is what separates a reactive tool caller from a true autonomous agent.

The Agent Loop Architecture



User Query

Agent Decision

Execute Tool

Re-evaluate

Final Answer

The loop runs continuously — decide, act, observe, re-decide — until the agent determines no further tool calls are needed and a final answer can be generated.

Sequential vs Parallel Execution

Sequential

Tool B waits for Tool A's result before executing.

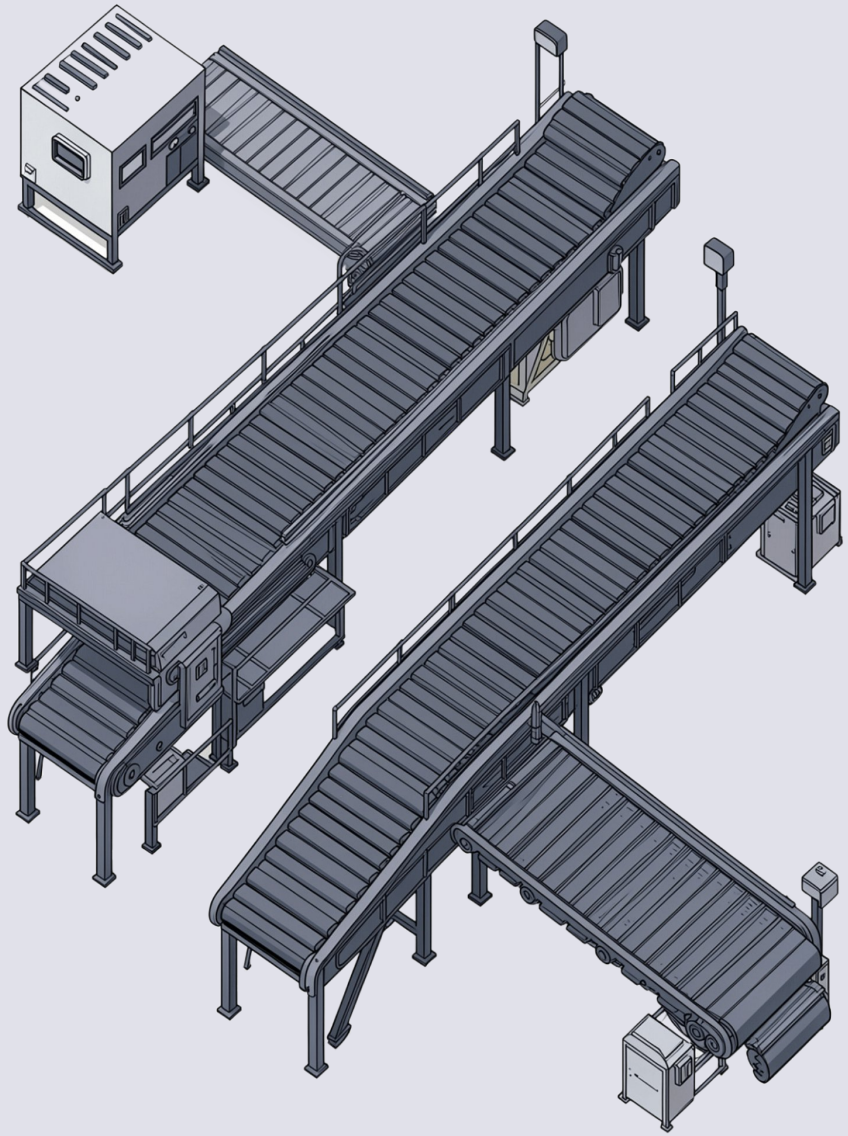
Database query → Search using product name → Calculate comparison

Parallel

Tools run independently; results are combined afterward.

Weather Pune + Weather Mumbai execute simultaneously

The agent infers execution strategy from **data dependencies** — not from explicit programmer instructions.



Building the Agent Loop: Core Pattern

```
while True:
    decision = get_agent_decision()

    if decision["type"] == "tool_call":
        result = execute_tool(
            decision["name"],
            decision["arguments"]
        )
        add_observation(result)
    else:
        break  # Final answer ready
```

The Decide → Execute → Observe Cycle

1 Decide

Agent evaluates current state and selects the next action

2 Execute

Named tool is called with agent-specified arguments

3 Observe

Result is appended to state — loop restarts

Agent State: Memory of Progress

What State Tracks

User Query

Original intent the agent is solving

Tool Calls Made

Which tools have already been invoked

Observations Received

Results accumulated from each tool execution

Final Answer

Populated only when the loop terminates

State in Action

After database call: `product = "Laptop"`, `revenue = ₹8,40,000`

After calculator call: `USD = $10,000`

Each subsequent decision is made with full awareness of what has already been retrieved.

⚠ State ≠ Conversation history. State is **workflow data** — structured context for the current task execution, not a chat log.

Example: Multi-Step Workflow

User: "Find highest-paid AI employee and convert salary to USD"

1

Database Query

Retrieve highest-paid AI employee → `₹12,00,000`

2

Calculator Call

Convert INR to USD (÷84) → `$14,285`

3

Generate Answer

Synthesize both observations into a final response

Each step depends on the previous observation — this is a purely sequential chain where no parallelism is possible.

Tool Registry Pattern

```
tools = {
    "calculator": calculator,
    "search":     search_web,
    "weather":    get_weather,
    "database":   execute_sql
}
```

Why a Registry?

Centralized Management

Add or remove tools without redesigning the agent loop

Explicit Allowlist

Only registered tools can be invoked — no surprises

Prevents Arbitrary Execution

LLM cannot call functions outside the registry boundary

Executing Tools Safely

```
def execute_tool(name, arguments):
    if name not in tools:
        raise ValueError("Unknown tool")

    return tools[name](**arguments)
```

Two lines. Two critical safety gates.

Safety Rules — Non-Negotiable

Validate Name

Reject any tool name not in the allowlist

Unpack Safely

Use `**arguments` with validated schema only

No Arbitrary Code

Never `eval()` or execute raw LLM output directly

Agent Termination: When to Stop

Every autonomous loop **must** have termination controls. Infinite loops are a production failure mode, not an edge case.

1

Final Answer Generated

Agent determines the task is complete → exit loop normally

2

MAX_STEPS Reached

Hard cap (e.g., 5 iterations) prevents infinite loops from runaway reasoning

3

Tool Failure Limit Exceeded

Repeated tool errors trigger a graceful failure report rather than silent looping

4

High-Risk Action Detected

Irreversible or sensitive operations pause execution and await human approval

⊗ An agent loop without termination controls is not a production system — it is a liability.